

Database Schema & Data Models

Introduction

The proposed AI Evaluation System uses a structured database design to efficiently manage user requests, uploaded documents, knowledge base chunks, AI-generated responses, evaluation results, and metadata. The database schema is designed to support Retrieval-Augmented Generation (RAG), semantic search, response evaluation, and future scalability.

The system uses **SQLite** as the relational database for storing structured application data and **FAISS** as the vector database for storing semantic embeddings used during document retrieval.

1. Collections / Tables

The system consists of the following primary tables.

1. Users

Stores user information.

Field	Data Type	Description
user_id	INTEGER	Primary Key
username	TEXT	User Name
email	TEXT	User Email
created_at	DATETIME	Registration Date

2. Documents

Stores uploaded documents.

Field	Data Type	Description
document_id	INTEGER	Primary Key
file_name	TEXT	Document Name
file_type	TEXT	PDF, DOCX, TXT
uploaded_at	DATETIME	Upload Time

Field	Data Type	Description
uploaded_by	INTEGER	User ID

3. Knowledge Base

Stores processed document chunks.

Field	Data Type	Description
chunk_id	INTEGER	Primary Key
document_id	INTEGER	Reference Document
chunk_text	TEXT	Chunk Content
embedding_id	TEXT	Vector Reference

4. Requests

Stores user queries.

Field	Data Type	Description
request_id	INTEGER	Primary Key
user_id	INTEGER	User ID
query	TEXT	User Question
timestamp	DATETIME	Request Time

5. Responses

Stores AI-generated responses.

Field	Data Type	Description
response_id	INTEGER	Primary Key
request_id	INTEGER	Linked Request
response_text	TEXT	Generated Response
generated_at	DATETIME	Generation Time

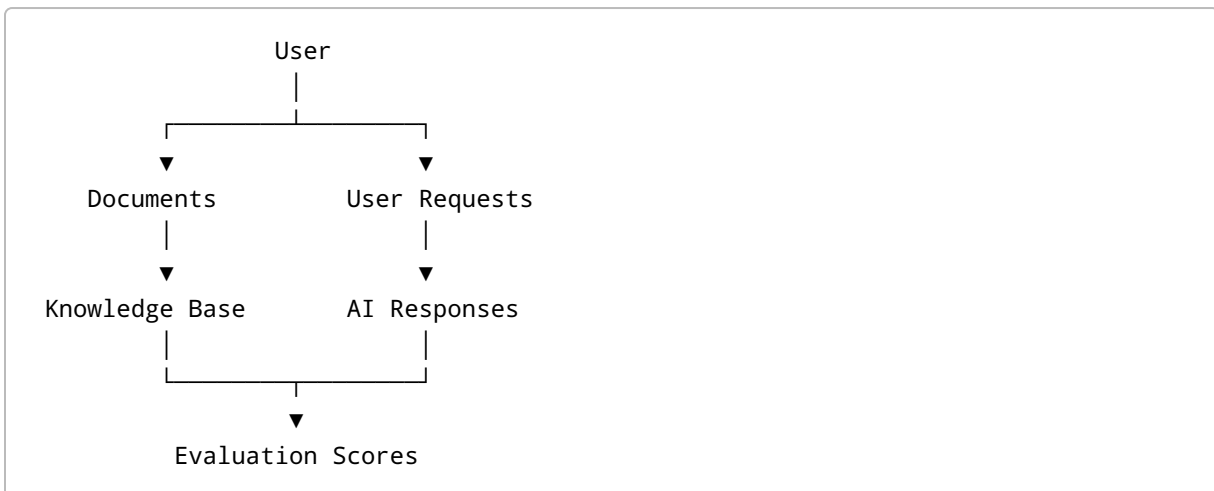
6. Evaluation Scores

Stores evaluation results.

Field	Data Type	Description
evaluation_id	INTEGER	Primary Key
response_id	INTEGER	Linked Response
relevance_score	FLOAT	Relevance Score
accuracy_score	FLOAT	Accuracy Score
hallucination_score	FLOAT	Hallucination Score
completeness_score	FLOAT	Completeness Score
overall_score	FLOAT	Final Score
verdict	TEXT	Excellent / Good / Average / Poor

2. Entity Relationships

The following diagram illustrates the relationship between the major entities.



Relationship Summary

- One user can upload multiple documents.
- One document contains multiple knowledge chunks.
- One user can submit multiple requests.
- One request generates one AI response.

- One AI response has one evaluation record.

3. Request Schema

Each user request follows the schema below.

Field	Description
request_id	Unique request identifier
user_id	User identifier
query	User question
timestamp	Request creation time

Sample Request

```
{
  "request_id": 101,
  "user_id": 1,
  "query": "Explain Machine Learning",
  "timestamp": "2026-07-03T10:30:00"
}
```

4. Response Schema

After processing the request, the AI system generates a response.

Field	Description
response_id	Unique response identifier
request_id	Related request
response_text	Generated answer
overall_score	Final evaluation score
verdict	Response quality

Sample Response

```
{
  "response_id": 501,
  "request_id": 101,
  "response_text": "Machine Learning is a branch of Artificial Intelligence...",
  "overall_score": 94,
  "verdict": "Excellent"
}
```

5. Knowledge Base Schema

The knowledge base stores processed document chunks that are used by the Retrieval-Augmented Generation (RAG) pipeline.

Field	Description
chunk_id	Unique chunk identifier
document_id	Parent document
chunk_text	Document content
embedding_id	Vector embedding reference
source_page	Page number

Sample Knowledge Base Record

```
{
  "chunk_id": 2501,
  "document_id": 12,
  "chunk_text": "Machine Learning is a subset of Artificial Intelligence...",
  "embedding_id": "emb_2501",
  "source_page": 8
}
```

6. Metadata

Metadata provides additional information that helps manage documents, responses, and evaluation results.

Document Metadata

- Document Name
- File Type
- File Size
- Upload Date
- Uploaded By
- Total Pages

Chunk Metadata

- Chunk Number
- Source Page
- Chunk Size
- Embedding Model

Response Metadata

- LLM Model (Gemini API)
- Response Time
- Token Count
- Evaluation Timestamp
- Overall Score
- Verdict

Metadata improves traceability, document management, and retrieval efficiency while supporting future analytics and auditing.

Database Design Summary

Component	Technology
Relational Database	SQLite
Vector Database	FAISS
Embedding Model	Sentence Transformers (all-MiniLM-L6-v2)
RAG Framework	LangChain
Large Language Model	Gemini API
Evaluation Frameworks	RAGAS + TruLens

Conclusion

The proposed database schema provides a well-structured foundation for managing user interactions, document knowledge, AI-generated responses, and evaluation results. SQLite efficiently stores structured application data, while FAISS enables high-speed semantic retrieval using vector embeddings. The schema supports the complete Retrieval-Augmented Generation workflow and integrates seamlessly with Gemini API, LangChain, RAGAS, and TruLens, ensuring scalability, maintainability, and reliable AI response evaluation.